

Constrained optimization benchmarks featuring a modified Newton Method

Michael Bartoloni

January 2026

1 Introduction

This is a project work for the Optimization Methods course (Università degli Studi di Firenze). It consists of the implementation and benchmarking of iterative optimization algorithms for constrained problems. In particular, the Projected Gradient Descent (PGD) and the constrained Newton Method are tested against a modified version of the latter, which consists of calculating the hessian matrix only on some iterations, aiming to reduce the computational overhead of such calculation while maintaining Newton's strong convergence properties. Relevant metrics are calculated to evaluate the performances of the three strategies on some PyCUTEst problems.

2 Implemented algorithms

All the implemented algorithms are based on the Armijo-type line search, adapted to solve constrained problems:

$$\min_{x \in S} f(x), \quad S \subseteq \mathbb{R}^n$$

Where the problem is subject to box constraints, *i.e.* $S = \{x \in \mathbb{R}^n \mid l_i \leq x_i \leq u_i \quad \forall i = 1, \dots, n\}$, $l, u \in \mathbb{R}^n$. With these constraints, S is guaranteed to be a convex set, and orthogonal projection of a point x can be easily implemented by clamping coordinates outside of the bounds:

$$z_i = \begin{cases} x_i & \text{if } x_i \in [l_i, u_i] \\ l_i & \text{if } x_i < l_i \\ u_i & \text{if } x_i > u_i \end{cases} \quad \text{for } z = P_S(x), \quad x \in S$$

The iterative framework follows this scheme:

$$x^{k+1} = x^k + \alpha_k d_k$$

Where α_k is an Armijo step size (Algorithm 1), and d_k is a descent direction. In a constrained setting, $d_k = \hat{x}^k - x^k$

Algorithm 1 Armijo line search

```
1: Input:  $x^k \in \mathbb{R}^n$ ,  $d_k \in \mathbb{R}^n$  :  $\nabla f(x^k)^T d_k < 0$ ,  $\Delta_0 > 0$ ,  $\gamma \in (0, 1)$ ,  $\delta \in (0, 1)$ 
2:  $\alpha = \Delta_0$ 
3: while  $f(x^k + \alpha d_k) > f(x^k) + \gamma \alpha \nabla f(x^k)^T d_k$  do
4:    $\alpha = \delta \alpha$ 
5: end while
6:  $\alpha_k = \alpha$ 
7: return  $\alpha_k$ 
```

2.1 Projected Gradient Descent

PGD finds $\hat{x}^k$ by solving

$$d_k = P_S(x^k - \nabla f(x^k)) - x^k = \hat{x}^k - x^k$$

The found direction is a descent direction, as seen in classes. The algorithm is sketched at Algorithm 2

Algorithm 2 Projected Gradient Descent

```
1: Input:  $x^0 \in S$ 
2:  $k = 0$ 
3: while  $\|x^k - P_S(x^k - \nabla f(x^k))\| \neq 0$  do
4:    $\hat{x}^k = P_S(x^k - \nabla f(x^k))$ 
5:    $d_k = \hat{x}^k - x^k$ 
6:   Find  $\alpha_k \in (0, 1]$  with an Armijo line search along  $d_k$ 
7:    $x^{k+1} = x^k + \alpha_k d_k$ 
8:    $k = k + 1$ 
9: end while
10: return  $x^k$ 
```

2.2 Constrained Newton Method

Leveraging second order information is not always possible, as the hessian must be positive definite to apply the method at all. In this setting, when $\nabla^2 f(x^k) \not\succ 0$, a PGD step is performed, with d_k as previously described. Instead, when $\nabla^2 f(x^k) \succ 0$, $\hat{x}^k$ is given by:

$$\hat{x}^k \in \arg \min_{x \in S} \nabla f(x^k)^T (x - x^k) + \frac{1}{2} (x - x^k)^T \nabla^2 f(x^k) (x - x^k) \quad (1)$$

With such point, we can derive $d_k = \hat{x}^k - x^k$, and perform a line search. Algorithm 3 summarizes the method.

2.3 Modified Newton Method

Calculating $\nabla^2 f(x^k)$ is costly, and doing it at each iteration may be too wasteful, not only in terms of computational resources, but also in terms of search direction efficiency: there may be instances where the matrix can still provide some benefit in the descent direction search, and so it could be used for more than one iteration. This motivates a

Algorithm 3 Constrained Newton Method

```
1: Input:  $x^0 \in S$ 
2:  $k = 0$ 
3: while  $\|x^k - P_S(x^k - \nabla f(x^k))\| \neq 0$  do
4:   if  $\nabla^2 f(x^k) \succ 0$  then
5:     find  $\hat{x}^k$  solving (1)
6:   else
7:      $\hat{x}^k = P_S(x^k - \nabla f(x^k))$ 
8:   end if
9:    $d_k = \hat{x}^k - x^k$ 
10:  Find  $\alpha_k \in (0, 1]$  with an Armijo line search along  $d_k$ 
11:   $x^{k+1} = x^k + \alpha_k d_k$ 
12:   $k = k + 1$ 
13: end while
14: return  $x^k$ 
```

variant of the Newton Method in which a matrix H_k is used for (1), with $H_{-1} = I$. Given a certain constant m for each iteration $k \geq 0$, the check for $\nabla^2 f(x^k) \succ 0$ is performed only if $k \bmod m = 0$, and if it succeeds, then $H_k = \nabla^2 f(x^k)$, otherwise $H_k = H_{k-1}$.

Note that until a "Newton region" is found, the method is actually performing PGD direction, and after reaching the region, the method takes Newton-style steps. Algorithm 4 shows the full procedure.

Algorithm 4 Modified Constrained Newton Method

```
1: Input:  $x^0 \in S$ 
2:  $k = 0$ 
3:  $H_{-1} = I$ 
4: while  $\|x^k - P_S(x^k - \nabla f(x^k))\| \neq 0$  do
5:    $H_k = \begin{cases} \nabla^2 f(x^k) & \text{if } \nabla^2 f(x^k) \succ 0 \text{ and } (k \bmod m) = 0 \\ H_{k-1} & \text{otherwise} \end{cases}$ 
6:    $\hat{x}^k = \begin{cases} x^k - P_S(x^k - \nabla f(x^k)) & \text{if } H_k = I \\ \text{Solution of (1) substituting } \nabla^2 f(x^k) \text{ with } H_k & \text{otherwise} \end{cases}$ 
7:    $d_k = \hat{x}^k - x^k$ 
8:   Find  $\alpha_k \in (0, 1]$  with an Armijo line search along  $d_k$ 
9:    $x^{k+1} = x^k + \alpha_k d_k$ 
10:   $k = k + 1$ 
11: end while
12: return  $x^k$ 
```

3 Experiments and results

To test the algorithms for actual performance, three different suites of PyCUTEst box constrained problems are selected:

- A group of 10 problems with objective label "*other*", aimed to test general constrained optimization problems in many different cases, varying n , the problem size.

- A group of 3 problems with objective label ”*quadratic*” or ”*sum of squares*”, aimed to test the methods with problems that can be more suited for Newton-style methods.
- 2 problems with objective label ”*other*”, with a good range of choice for values of n . These should provide a fair comparison for the scaling of the algorithms under the same objective.

For each problem and algorithm the tests measure:

1. The optimal value of the objective function.
2. The number of iterations to reach a solution. For Newton methods, also the number of Newton iterations is recorded (for the modified method, it simply marks the first time the hessian is found to be positive definite).
3. The execution time taken to reach a stationary point.

3.1 Practical details

- To avoid a very long runtime on large problems, the maximum number of iterations is set to 10000 for all methods.
- The stopping condition for stationary points accepts a tolerance of $\epsilon = 10^{-3}$.
- The tests for the Modified Newton Method feature different m values, ranging from 5 to 40, with a step of 5.
- For the alternative objective problems, BQPGASIM and JNLBRNG1 are ”quadratic”, HS110 is ”sum of squares”.
- For the Modified Newton Method, there may be cases in which the old Hessian ends up in a region of very low descent. If in this region the Hessian is also not positive definite, and the stationary condition is not reached, the method could run infinitely (or until the maximum iteration number is reached), so a fallback step to PGD is added when the norm of the direction is very low, i.e. $\|d^k\| \leq 10^{-8}$. This is meant to mitigate the effect, and reduce runtime, especially on highly-dimensional problems.
- The solver for the subproblem is *cvxopt*, and sometimes some numerical errors occur due to floating point precision (especially divisions by 0). To overcome these, each bound is checked for its magnitude, and if it’s too high or low, it gets removed. In fact, PyCUTEst models unconstrained upper or lower bounds with 10^{20} and 10^{-20} respectively, so the eventual loss of precision introduced in this step is negligible.
- To suit the *cvxopt* quadratic problem model, plugging (1) requires some rearrangement expanding the quadratic formula, exploiting the symmetry of $\nabla^2 f(x^k)$ and eliminating the constant term, obtaining

$$\hat{x}^k \in \arg \min_{x \in S} (\nabla f(x^k) - \nabla^2 f(x^k)x^k)^T x + \frac{1}{2}x^T \nabla^2 f(x^k)x$$

Which is an equivalent standard form for the quadratic problem.

Problem (N)	PGD	Newton	MN-m5	MN-m10	MN-m15	MN-m20	MN-m25	MN-m30	MN-m35	MN-m40
BRANIN (2)	3.98e-01	3.98e-01	3.98e-01	3.98e-01	3.98e-01	3.98e-01	3.98e-01	3.98e-01	3.98e-01	3.98e-01
DGOSPEC (3)	-9.97e+02	-9.97e+02	-9.97e+02	-9.97e+02	-9.97e+02	-9.97e+02	-9.97e+02	-9.97e+02	-9.97e+02	-9.97e+02
HS38 (4)	1.02e-07	7.71e+00	4.07e-13	1.14e-10	1.18e-12	7.27e+00	1.38e-09	6.86e-10	1.27e-09	2.21e-13
HS45 (5)	1.73e+00	1.73e+00	1.73e+00	1.73e+00	1.73e+00	1.73e+00	1.73e+00	1.73e+00	1.73e+00	1.73e+00
HART6 (6)	-3.32e+00	-3.32e+00	-3.32e+00	-3.32e+00	-3.32e+00	-3.32e+00	-3.32e+00	-3.32e+00	-3.32e+00	-3.32e+00
S368 (8)	-7.50e-01	-7.50e-01	-7.50e-01	-7.50e-01	-7.50e-01	-7.50e-01	-7.50e-01	-7.50e-01	-7.50e-01	-7.50e-01
HOLMES (180)	1.25e+03	1.25e+03	1.25e+03	1.25e+03	1.25e+03	1.25e+03	1.25e+03	1.25e+03	1.25e+03	1.25e+03
HADAMALS (380)	7.16e+03	7.16e+03	7.16e+03	7.16e+03	7.16e+03	7.16e+03	7.16e+03	7.16e+03	7.16e+03	7.16e+03
PROBPENL (500)	3.99e-07	3.99e-07	3.99e-07	3.99e-07	3.99e-07	3.99e-07	3.99e-07	3.99e-07	3.99e-07	3.99e-07
SINEALI (1000)	-9.98e+04	-9.98e+04	-9.98e+04	-9.98e+04	-9.98e+04	-9.98e+04	-9.98e+04	-9.98e+04	-9.98e+04	-9.98e+04

Table 1: Objective value for the approximate stationary points in constrained problems with objective ”*other*”

Problem (N)	PGD	Newton	MN-m5	MN-m10	MN-m15	MN-m20	MN-m25	MN-m30	MN-m35	MN-m40
BRANIN (2)	36	5 (4)	10 (5)	14 (4)	18 (3)	22 (2)	27 (2)	32 (2)	36 (1)	36 (0)
DGOSPEC (3)	10000	10000 (9999)	10000 (9994)	10000 (9983)	10000 (9972)	10000 (9962)	10000 (9952)	10000 (9942)	10000 (9932)	10000 (9922)
HS38 (4)	1296	10000 (7)	137 (137)	443 (443)	272 (272)	10000 (10000)	4238 (4238)	3670 (3670)	3400 (3400)	3122 (3122)
HS45 (5)	10000	10000 (0)	10000 (0)	10000 (0)	10000 (0)	10000 (0)	10000 (0)	10000 (0)	10000 (0)	10000 (0)
HART6 (6)	30	9 (6)	12 (7)	13 (3)	17 (2)	22 (2)	27 (2)	30 (0)	30 (0)	30 (0)
S368 (8)	29	29 (0)	29 (0)	29 (0)	29 (0)	29 (0)	29 (0)	29 (0)	29 (0)	29 (0)
HOLMES (180)	1485	4 (4)	6 (6)	6 (6)	6 (6)	6 (6)	6 (6)	6 (6)	6 (6)	6 (6)
HADAMALS (380)	862	862 (0)	862 (0)	862 (0)	862 (0)	862 (0)	862 (0)	862 (0)	862 (0)	862 (0)
PROBPENL (500)	64	64 (0)	64 (0)	64 (0)	64 (0)	64 (0)	64 (0)	64 (0)	64 (0)	64 (0)
SINEALI (1000)	498	22 (15)	48 (34)	56 (40)	75 (54)	111 (85)	135 (79)	141 (105)	222 (146)	218 (172)

Table 2: Number of iterations for constrained problems with objective ”*other*”

Problem (N)	PGD	Newton	MN-m5	MN-m10	MN-m15	MN-m20	MN-m25	MN-m30	MN-m35	MN-m40
BRANIN (2)	0.0032	0.0041	0.0026	0.0029	0.0032	0.0031	0.0034	0.0041	0.0041	0.0037
DGOSPEC (3)	4.4337	6.7853	17.1327	17.5952	17.6086	17.5294	17.6997	17.4968	17.5147	17.3213
HS38 (4)	0.3421	3.1611	0.0543	0.2098	0.1369	6.4458	1.7067	1.4985	1.3953	1.3217
HS45 (5)	15.5648	16.4905	15.1007	15.0141	15.4866	15.1988	15.3439	15.1598	15.2046	15.2048
HART6 (6)	0.0045	0.0041	0.0042	0.0029	0.0034	0.0040	0.0050	0.0051	0.0050	0.0050
S368 (8)	0.0008	0.0024	0.0021	0.0017	0.0015	0.0017	0.0015	0.0014	0.0014	0.0015
HOLMES (180)	21.7963	1.6293	2.6996	2.4298	2.4559	2.4059	2.4138	2.3607	2.4274	2.4487
HADAMALS (380)	0.7518	19.4518	6.6210	4.4174	3.8358	3.6497	3.2787	3.0950	2.9786	2.8620
PROBPENL (500)	0.0739	11.2370	2.4204	1.3876	1.0767	0.9124	0.7201	0.7093	0.5467	0.5486
SINEALI (1000)	1.0690	14.4034	8.8585	7.1444	8.1708	12.5151	11.9256	14.5329	19.7806	22.5558

Table 3: Execution time (s) for constrained problems with objective ”*other*”

3.2 Results

Tables 1, 2, 3 show the results for the set of ”*other*” objective problems. First, on some problems (DGOSPEC, HS45) a stationary point is not reached in 10000 steps for all methods. Maybe their topology is hard for the algorithms, and it could be the case that the direction does not bring the next point much far from the current one for each iteration. Also, Newton-style methods sometimes do not converge on HS38, and probably this is due to some bad Newton steps that end up in almost flat or regions with no Hessian support, hence why the Newton Method performs almost all steps as PGD, or a bad local minimum. There are four visible outcomes for the methods applied to the problems:

- The methods converge and the sequence of generated points enters a region in which the Hessian matrix is positive definite and brings serious benefits in terms of iterations
- The methods converge but the sequence of generated points never finds the Hessian matrix to be positive semidefinite, so Newton-style methods take only PGD steps, with the added cost of calculating the Hessian, and checking if it is positive definite.
- The methods converge with PGD, but Newton-style methods may not converge, because some step brings them into a suboptimal region they cannot exit from.

- The methods do not converge, regardless of the positive definiteness of the Hessian matrix.

Looking at the optimal values across the different problems, the Newton-style methods seem to perform at least equally with respect to PGD when they converge, if not much better in the case of HS38.

When n is low, the Modified Newton Method actually outperforms PGD and Newton Method (if all converge), for a low value of m (5 or 10), while as n grows, computing $\nabla^2 f(x^k) \succ 0$ becomes computationally expensive, and PGD becomes more efficient, even when making more steps, as shown in the results for SINEALI. Also, with large n , solving (1) becomes expensive, so making more iterations can be actually detrimental to performance, so it's critical to not pick a value for m that is excessively large.

Tables 4, 5, 6 show the results for the alternative objectives set of problems. Here,

Problem (N)	PGD	Newton	MN-m5	MN-m10	MN-m15	MN-m20	MN-m25	MN-m30	MN-m35	MN-m40
JNLBRNG1 (4)	-2.25e-01	-2.25e-01	-2.25e-01	-2.25e-01	-2.25e-01	-2.25e-01	-2.25e-01	-2.25e-01	-2.25e-01	-2.25e-01
HS110 (10)	-4.58e+01	-4.58e+01	-4.58e+01	-4.58e+01	-4.58e+01	-4.58e+01	-4.58e+01	-4.58e+01	-4.58e+01	-4.58e+01
BQPGASIM (50)	-5.50e-05	-5.52e-05	-5.52e-05	-5.52e-05	-5.52e-05	-5.52e-05	-5.52e-05	-5.52e-05	-5.52e-05	-5.52e-05

Table 4: Objective value for the approximate stationary points in the alternative objective problems

Problem (N)	PGD	Newton	MN-m5	MN-m10	MN-m15	MN-m20	MN-m25	MN-m30	MN-m35	MN-m40
JNLBRNG1 (4)	21	2 (2)	2 (2)	2 (2)	2 (2)	2 (2)	2 (2)	2 (2)	2 (2)	2 (2)
HS110 (10)	24	6 (6)	9 (9)	12 (12)	17 (17)	22 (22)	26 (26)	26 (26)	26 (26)	26 (26)
BQPGASIM (50)	2802	2 (2)	2 (2)	2 (2)	2 (2)	2 (2)	2 (2)	2 (2)	2 (2)	2 (2)

Table 5: Number of iterations for the alternative objective problems

Problem (N)	PGD	Newton	MN-m5	MN-m10	MN-m15	MN-m20	MN-m25	MN-m30	MN-m35	MN-m40
JNLBRNG1 (4)	0.0017	0.0014	0.0013	0.0012	0.0013	0.0012	0.0012	0.0012	0.0012	0.0012
HS110 (10)	0.0021	0.0054	0.0063	0.0081	0.0094	0.0122	0.0143	0.0144	0.0153	0.0148
BQPGASIM (50)	0.9720	0.0210	0.0053	0.0058	0.0045	0.0057	0.0045	0.0057	0.0045	0.0059

Table 6: Execution time (s) for the alternative objective problems

the Newton-style methods perform way better in terms of iterations, and also on a quadratic problem more difficult for PGD (BQPGASIM), manages to reach a solution in just 2 iterations, cutting execution time by a wide margin with respect to PGD. Modified Newton performs a little better in terms of time in quadratic problems, but for HS110, which has a sum of squares objective, it struggles to find a solution, and for $m \geq 25$, it takes more steps than PGD, meaning it's less efficient.

Figures 1, 2 plot the execution time varying n for CHARDIS0 and HADAMALS, in logarithmic scale to better highlight differences for lower execution times.

No Newton steps are taken in solving HADAMALS, as seen in Table 2, so clearly Newton-style methods are only adding computations to the results, effectively degrading performance. However, CHARDIS0 shows that second order information has some advantage for low n : all Newton-style methods outperform PGD, until the extra computation becomes too high for each iteration, dominating the execution time.

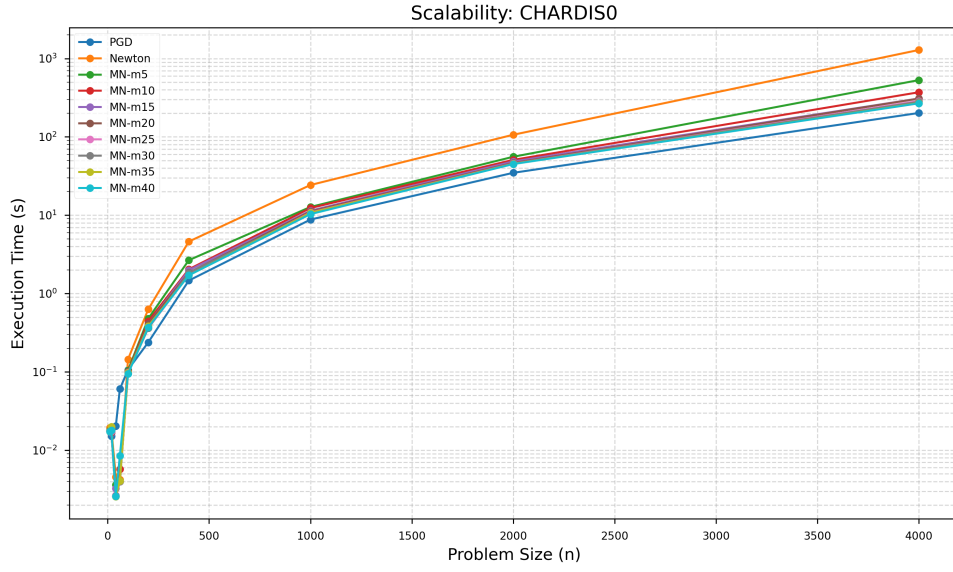


Figure 1: CHARDISO execution time (s) varying the size of the problem

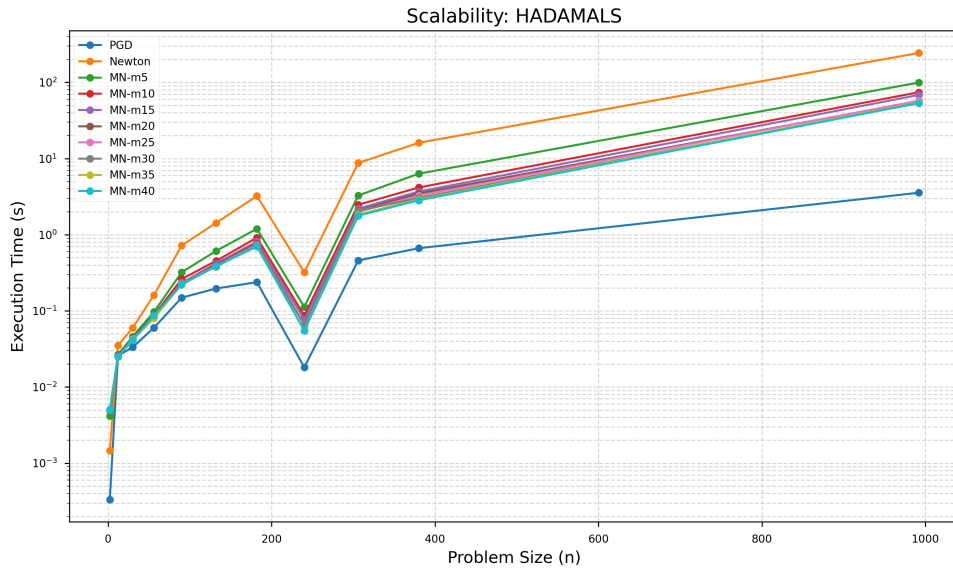


Figure 2: HADAMALS execution time (s) varying the size of the problem

4 Conclusions

The Modified Newton Method can be very useful for two main reasons:

1. Keeping the old Hessian matrix for some iterations can avoid some bad steps due to the topology of the problems, pushing through and stabilizing the descent
2. Reducing the computational load when the problem is regular enough that reusing the Hessian for some iterations can bring to more benefits than losses of performance due to the increased iterations

Unfortunately, these highly depend on the type of the problem, and for a high n it's just better to stick with PGD. But if the problem is suited for a Newton approach, a choice of m should not be too big (around 20) because too many added iterations brings to more expensive quadratic subproblem calculations. For smaller and medium sized problems, a good choice of m should be as small as possible (5-10 should be good), since it's better to update H_k more frequently to get better information for the descent direction, as checking $\nabla^2 f(x^k)$ is not as costly. It is to be noted that for larger scale quadratic problems Modified Newton works much better than both PGD and Newton, since convergence can be reached with less Hessian computations than Newton, but with a similar number of iterations.